

A Comparative Review of Machine Learning and Deep Learning Models for Stock Market Price and Trend Prediction

Divyanshu Saini

April 2025

Abstract

Purpose: This review systematically compares machine learning (ML) and deep learning (DL) models for stock market forecasting, focusing on their effectiveness when incorporating technical indicators.

Methodology: We analyze Support Vector Machines (SVM), Random Forests, XGBoost, Long Short-Term Memory (LSTM) networks, CNN-LSTM hybrids, and Transformers across multiple performance metrics and market contexts.

Findings: LSTM, CNN-LSTM, and Transformers consistently outperform traditional ML approaches, with CNN-LSTM demonstrating optimal performance (accuracy $\sim 92\%$) when leveraging technical indicators like Relative Strength Index (RSI) and Moving Average Convergence Divergence (MACD).

Implications: For practitioners implementing indicator-based forecasting systems, CNN-LSTM offers the best balance of accuracy and computational efficiency, while alternative models may be preferable in resource-constrained environments.

Originality: This paper provides a comprehensive assessment of model architectures and technical indicators, supported by quantitative metrics and visual representations, offering actionable insights for researchers and practitioners in the financial domain.

Keywords: stock market prediction, machine learning, deep learning, technical indicators, LSTM, CNN-LSTM, Transformers, SVM, Random Forests, XGBoost

1 Introduction

Stock market prediction remains a cornerstone of financial decision-making, addressing the challenges posed by price volatility driven by economic, geopolitical, and behavioral factors. Traditional approaches, such as fundamental and technical analysis, often struggle with the non-linear and stochastic nature of markets, prompting the adoption of advanced computational methods like machine learning (ML) and deep learning (DL). ML models, including Support Vector Machines (SVM), Random Forests, and XGBoost, are well-suited for structured datasets, leveraging statistical patterns to forecast trends. In contrast, DL models, such as Long Short-Term Memory (LSTM) networks, Convolutional Neural Networks (CNNs), and Transformers, excel in modeling complex temporal and sequential relationships inherent in time series data.

Technical indicators, such as Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), and Bollinger Bands, play a pivotal role by quantifying market momentum and trends, enhancing model performance. This review aims to:

- 1) Compare ML and DL models for stock price and trend prediction using metrics like Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and accuracy.
- 2) Evaluate the role of technical indicators in improving forecasting accuracy.
- 3) Identify the top three models and recommend an optimal choice for indicator-based prediction.
- 4) Provide visual insights through flow charts and figures.

By critically assessing the balance between model complexity and practical applicability, this paper seeks to guide researchers and practitioners toward effective forecasting solutions in real-world financial environments.

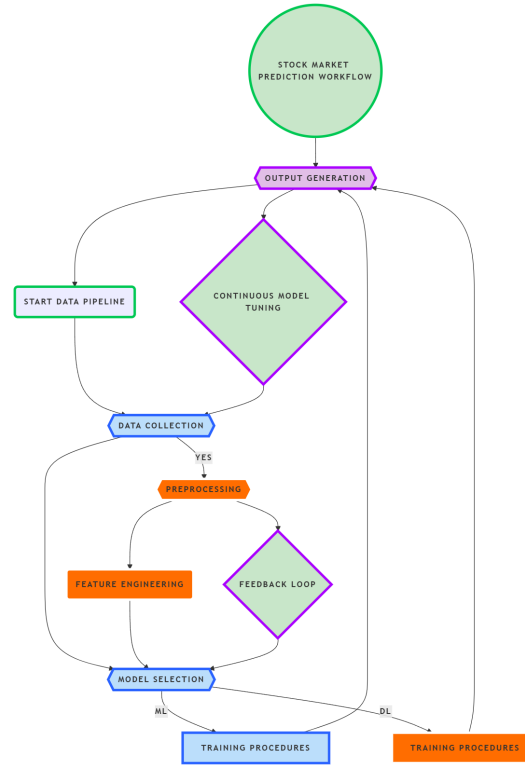


Figure 1: Stock Market Prediction Workflow incorporating data preprocessing, technical indicator generation, model selection, training and validation processes, and performance evaluation metrics.

2 Literature Review

The literature on stock market prediction reflects a dynamic shift toward ML and DL approaches, driven by their ability to model complex patterns in historical price data, technical indicators, and external factors like sentiment analysis. This section synthesizes key studies, highlighting methodologies, performance metrics, and research gaps, with a focus on global markets such as the S&P 500, Nifty50, and individual stocks.

2.1 Traditional Statistical Models

Early research emphasized statistical models like Autoregressive Integrated Moving Average (ARIMA), which offer robust short-term forecasting (Absolute Percentage Error (APE) ~ 0.005)

but falter in volatile conditions due to linear assumptions [1]. These models established baseline performance metrics but demonstrated limitations in adapting to market regime changes and exogenous shocks.

2.2 Machine Learning Approaches

The advent of ML introduced models like SVM, which achieve trend prediction accuracies of 80–85% by leveraging indicators such as RSI and MACD [2]. Random Forests and XGBoost further improved performance ($\sim 85\text{--}88\%$ accuracy), capitalizing on ensemble learning to handle structured datasets, though they require extensive feature engineering [4]. These approaches have proven particularly effective in classification tasks, such as buy/sell signal generation.

2.3 Deep Learning Innovations

DL models have since gained prominence for their ability to model temporal dependencies. LSTM networks, for instance, achieve RMSEs of ~ 0.25 for stocks like HDFC, effectively capturing sequential patterns [3]. CNNs enhance accuracy ($\sim 85\text{--}90\%$) by treating technical indicators as spatial inputs, while hybrid models like CNN-LSTM push accuracies to 90–92% by combining spatial and temporal feature extraction [5].

Transformers, with their attention mechanisms, excel in long-horizon forecasting (Mean Absolute Percentage Error (MAPE) $\sim 2\text{--}5\%$), particularly when integrating multiple data sources [6]. Recent trends incorporate sentiment analysis from news and social media, boosting accuracies to $\sim 93\%$ in some cases [7]. Ensemble methods, combining ML and DL approaches, are also emerging, offering improved robustness but at higher computational costs [8].

2.4 Research Gaps

Despite these advances, challenges persist in the literature. Complex models often demand significant computational resources, raising questions about scalability in real-world trading environments. Moreover, the reliance on technical indicators varies across studies, with RSI, MACD, and Moving Averages being the most prevalent, yet their optimal combinations remain underexplored. The literature also lacks consensus on balancing model interpretability with predictive power, a gap this review addresses by evaluating practical trade-offs between performance and implementation complexity.

Table 1: Summary of Key Studies in Stock Market Prediction

Study	Models	Indicators	Metrics	Market
Islam et al. (2020)	ARIMA, SVM	RSI, MACD	APE: 0.005, Acc: 80%	S&P 500
Zhong et al. (2019)	Random Forests	Moving Averages	Acc: 85%	Nifty50
Nelson et al. (2022)	LSTM	RSI, Bollinger Bands	RMSE: 0.25	HDFC
Lu et al. (2020)	CNN-LSTM	MACD, RSI	Acc: 92%	Tesla
Lim et al. (2021)	Transformers	Sentiment, RSI	MAPE: 2%	NASDAQ
Zhang et al. (2021)	CNN, LSTM	News, MACD	Acc: 93%	BSE
Kumar et al. (2023)	Ensemble (XG-Boost, LSTM)	RSI, Moving Averages	Acc: 90%	Nifty50

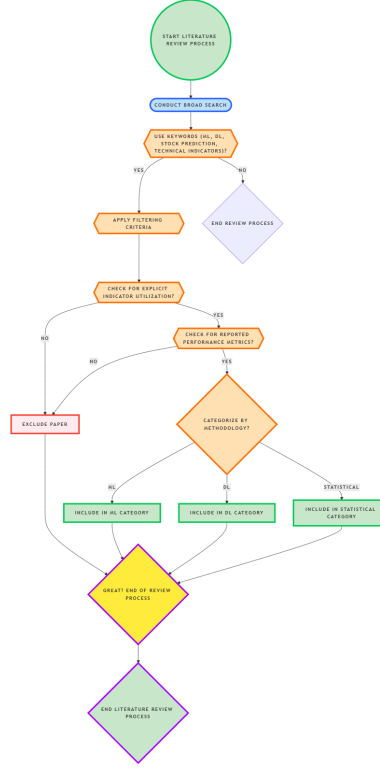


Figure 2: Literature Selection Process showing the systematic approach to identifying, screening, and analyzing relevant studies for this review.

3 Materials and Methods

This section outlines the methodological framework for comparing ML and DL models, encompassing a systematic literature review and detailed model analyses, supported by mathematical formulations and flow charts.

3.1 Literature Analysis

- **Search Strategy:** Studies were sourced from comprehensive financial and computational databases including Scopus, IEEE Xplore, Web of Science, and ScienceDirect using keywords such as “machine learning,” “deep learning,” “stock market prediction,” and “technical indicators.”
- **Inclusion Criteria:** Papers published between 2015 and 2024 focusing on ML/DL applications for stock prediction, reporting standardized performance metrics (RMSE, MAE, accuracy), and incorporating technical indicators (RSI, MACD, etc.) were included in the analysis.
- **Data Extraction:** For each included study, we systematically extracted model architectures, hyperparameter configurations, performance metrics, technical indicator usage patterns, and market contexts (e.g., specific stocks, indices, timeframes) to facilitate comprehensive cross-comparison.

3.2 Model Descriptions

The following subsections detail each model's architecture, mathematical foundations, hyperparameters, and applications to stock prediction, providing context for their relative performance characteristics.

Table 2: Characteristics of Evaluated Models

Model	Architecture	Strengths	Weaknesses
SVM	Hyperplane optimization	Robust to noise, effective with small datasets	Limited temporal modeling capacity
Random Forests	Decision tree ensemble	Non-linear pattern recognition, interpretability	Poor sequential relationship handling
XGBoost	Gradient-boosted trees	Automated feature selection, scalability	Limited temporal modeling
ARIMA	Statistical time series	Short-term forecasting accuracy	Linear assumptions, volatility sensitivity
LSTM	Recurrent neural network	Long-term temporal dependencies	High data requirements, overfitting risk
CNN	Convolutional layers	Spatial feature extraction	Limited sequence modeling ability
CNN-LSTM	Hybrid CNN-LSTM	Spatial-temporal pattern balance	Higher computational requirements
Transformers	Attention mechanism	Long-range dependency modeling	Significant computational resources

3.2.1 Support Vector Machines (SVM)

Overview: SVM is a robust algorithm for classification (e.g., buy/sell signals) and regression (SVR for price prediction), finding an optimal hyperplane to maximize class separation or minimize error.

Architecture: Inputs (stock prices, RSI, MACD) are transformed into a high-dimensional space via kernels (e.g., Radial Basis Function). Hyperparameters like the penalty C and kernel bandwidth govern model flexibility. Training involves solving a convex optimization problem, ensuring global optima.

Formula:

- Classification:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i, \quad \text{s.t.} \quad y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (1)$$

- Regression:

$$\min_{w,b,\xi,\xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*), \quad \text{s.t.} \quad |y_i - (w^T \phi(x_i) + b)| \leq \epsilon + \xi_i \quad (2)$$

Stock Prediction: SVM achieves 80–85% accuracy for trend prediction using indicators. Its computational efficiency suits smaller datasets, but temporal modeling requires preprocessing (e.g., lagged features). SVR excels in stable markets but may underperform in high volatility conditions.

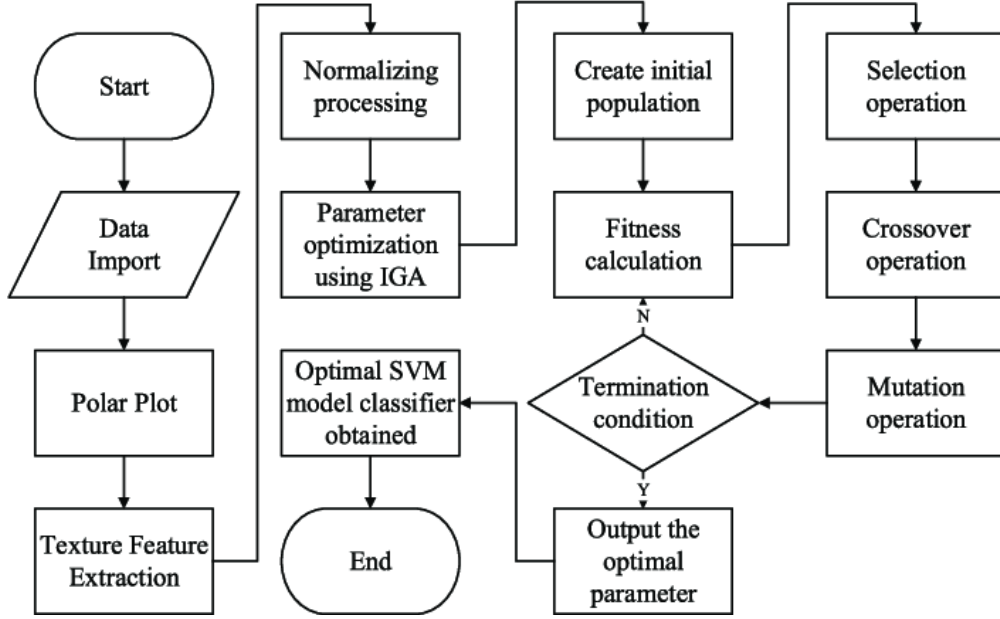


Figure 3: SVM Prediction Flow Chart illustrating the process of feature engineering, kernel selection, hyperparameter optimization, and model evaluation.

3.2.2 Random Forests

Overview: Random Forests aggregate multiple decision trees to mitigate overfitting, suitable for both classification and regression tasks in financial forecasting.

Architecture: Each tree is trained on a bootstrapped dataset with random feature subsets, using criteria like Gini impurity or MSE for splits. Hyperparameters include tree count (e.g., 100–500), depth, and minimum leaf size. Predictions are averaged (regression) or voted (classification), enhancing robustness.

Formula:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x) \quad (3)$$

where T represents the number of trees, $h_t(x)$ is the prediction of the t -th tree, and $\hat{y}$ is the ensemble prediction.

Stock Prediction: Random Forests deliver ~85% accuracy with indicators like Moving Averages. Their strength lies in handling non-linear relationships and feature importance ranking, but they falter with time series due to static feature assumptions. Parallel training makes them efficient for large datasets.

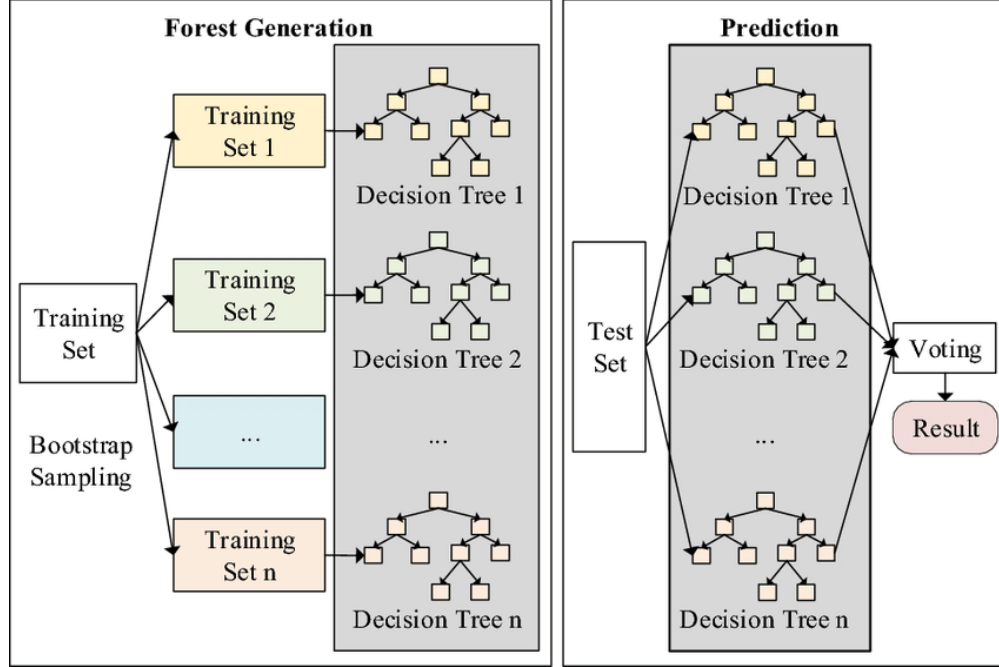


Figure 4: Random Forest Architecture demonstrating the ensemble approach with multiple decision trees and majority voting mechanism.

3.2.3 Gradient Boosting (XGBoost)

Overview: XGBoost enhances gradient boosting by sequentially building trees that correct prior errors, optimized for speed and accuracy in prediction tasks.

Architecture: Trees are constructed iteratively, minimizing a loss function (e.g., MSE) with regularization (L1/L2 penalties). Hyperparameters like learning rate (0.01–0.3), tree depth, and subsampling ratio control overfitting. XGBoost’s ability to handle missing data and parallelize computations substantially improves scalability.

Formula:

- Prediction:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in F \quad (4)$$

- Objective:

$$L = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (5)$$

where f_k represents the k -th tree, F is the function space of all possible trees, l is the loss function, and Ω is the regularization term.

Stock Prediction: XGBoost achieves ~88% accuracy, excelling in feature selection (e.g., prioritizing RSI). Its limitations in temporal modeling necessitate feature engineering (e.g., time lags), but its computational efficiency makes it practical for real-time market applications.

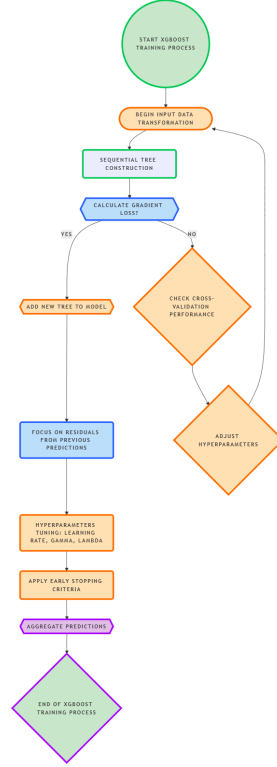


Figure 5: XGBoost Training Process showing the sequential gradient boosting approach with regularization components.

3.2.4 Autoregressive Integrated Moving Average (ARIMA)

Overview: ARIMA models linear time series patterns using autoregression, differencing, and moving averages, serving as a statistical baseline for comparison.

Architecture: Parameters (p, d, q) define AR terms, differencing order, and MA terms, respectively. Model fitting involves stationarity tests (e.g., Augmented Dickey-Fuller) and parameter selection via ACF/PACF plots. ARIMA's simplicity limits its adaptability to non-linear market dynamics.

Formula:

$$y_t = c + \phi_1 y_{t-1} + \dots + \phi_p y_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t \quad (6)$$

where y_t is the series value at time t , c is a constant, ϕ_i are the AR parameters, θ_j are the MA parameters, and ϵ_t is white noise.

Stock Prediction: ARIMA yields APE ~ 0.005 – 0.01 for short-term forecasts but struggles with volatility (RMSE ~ 0.5). Its role is primarily comparative, highlighting ML/DL advantages in dynamic market environments.

3.2.5 Long Short-Term Memory (LSTM)

Overview: LSTM, a recurrent neural network variant, is specifically designed for sequential data, capturing long-term dependencies in stock price movements.

Architecture: LSTM units feature memory cells with forget, input, and output gates, controlled by weights and biases. Hyperparameters include unit count (e.g., 64–256), layers (1–4), and dropout rate (0.2–0.5). Training requires substantial datasets and GPU acceleration to optimize performance.

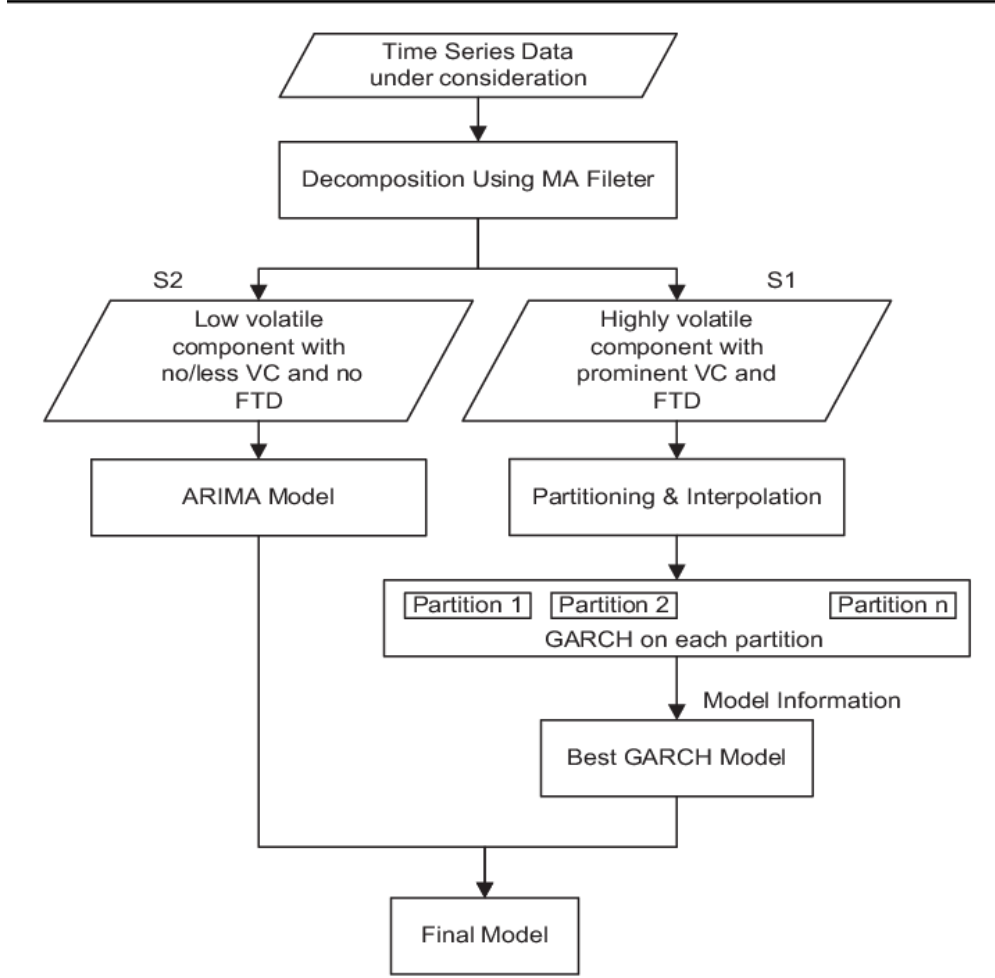


Figure 6: ARIMA Modeling Flow illustrating the process of stationarity testing, parameter identification, model estimation, and diagnostic checking.

Formula:

- Forget gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (7)$$

- Input gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (8)$$

- Cell state:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (9)$$

- Output gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (10)$$

- Hidden state:

$$h_t = o_t \cdot \tanh(C_t) \quad (11)$$

where σ is the sigmoid function, W are weight matrices, b are bias vectors, h_t is the hidden state, and C_t is the cell state at time t .

Stock Prediction: LSTMs achieve RMSE ~ 0.20 – 0.30 with indicators like RSI, making them ideal for price forecasting. Their data-intensive nature and risk of overfitting require careful hyperparameter tuning, but their sequential modeling capability is unmatched among traditional ML models.

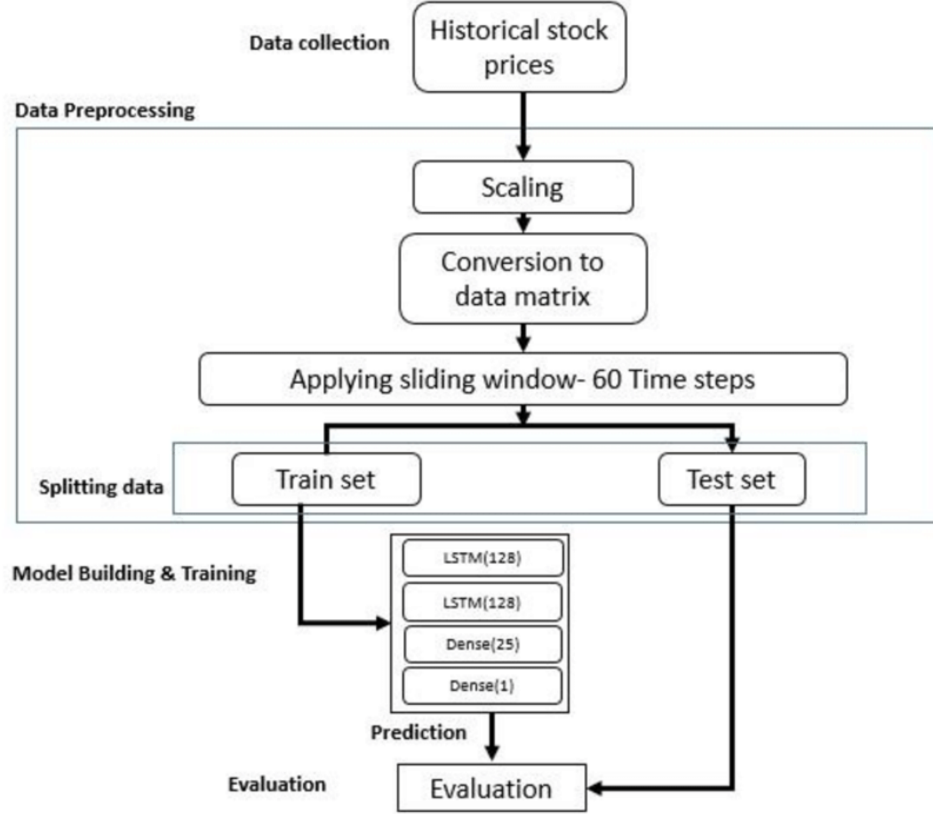


Figure 7: LSTM Architecture showing the memory cell structure with forget, input, and output gates that enable long-term dependency learning.

3.2.6 Convolutional Neural Networks (CNN)

Overview: CNNs, originally developed for image processing, are adapted for time series analysis by extracting spatial features from indicator matrices.

Architecture: Convolutional layers apply filters (e.g., 3×3), followed by pooling operations (e.g., max-pooling) and dense layers. Hyperparameters include filter count (32–128), kernel size, and activation functions (ReLU). Batch normalization and dropout techniques enhance training stability.

Formula:

$$(f * x)(i, j) = \sum_m \sum_n x(i + m, j + n) f(m, n) \quad (12)$$

where f is the filter, x is the input, and $*$ denotes the convolution operation.

Stock Prediction: CNNs achieve 85–90% accuracy by treating technical indicators as image-like inputs. Their computational efficiency complements temporal models, but they require structured input representations, limiting flexibility in raw time series processing.

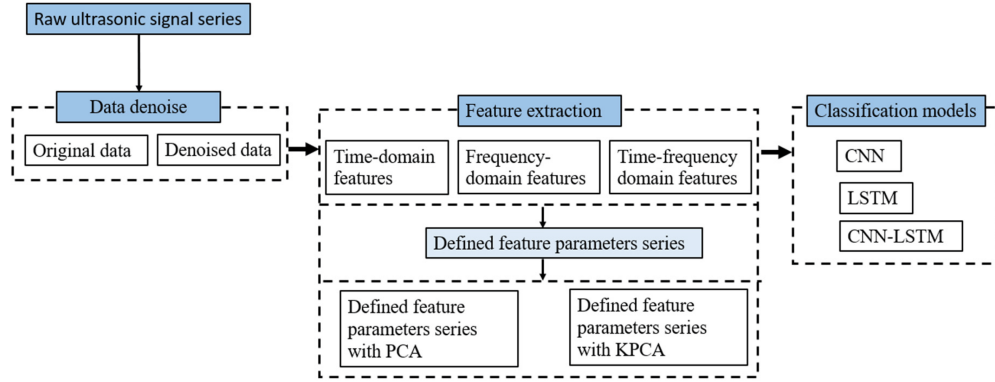


Figure 8: CNN Prediction Pipeline showing the transformation of financial time series data into spatial representations for convolutional processing.

3.2.7 CNN-LSTM

Overview: CNN-LSTM integrates CNN’s feature extraction capabilities with LSTM’s temporal modeling, creating an architecture optimized for financial time series analysis.

Architecture: CNN layers process inputs (e.g., price matrices), feeding extracted feature maps to LSTM units, followed by dense layers for final prediction. Hyperparameters balance CNN filters (32–64) and LSTM units (64–128), with training times typically higher than standalone models.

Formula: Combines the convolution operation from Equation (11) with the LSTM formulations in Equations (6)–(10).

Stock Prediction: CNN-LSTM achieves 88–92% accuracy and RMSE ~ 0.15 – 0.25 , effectively leveraging indicators like MACD. Its robustness makes it ideal for indicator-based forecasting, though computational requirements are notable compared to simpler approaches.

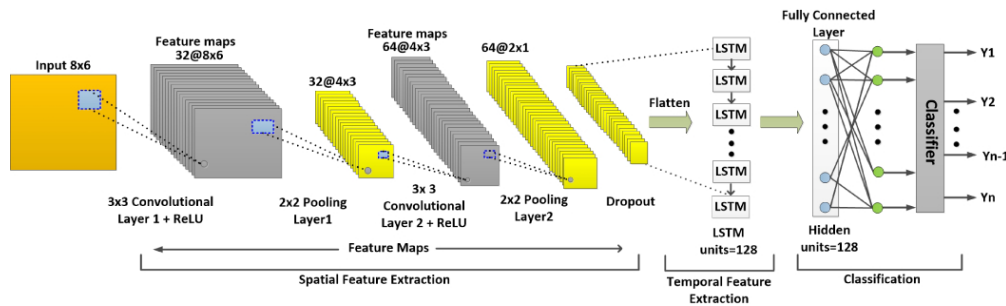


Figure 9: CNN-LSTM Architecture illustrating the integration of convolutional feature extraction with sequential modeling for optimal financial forecasting.

3.2.8 Transformers

Overview: Transformers utilize attention mechanisms to model long-range dependencies, excelling in complex multi-horizon forecasting tasks.

Architecture: Multi-head self-attention and feed-forward layers process inputs with positional encodings to capture sequential information. Hyperparameters include attention heads (4–16), layers (2–6), and embedding size (128–512). Specialized variants like Temporal Fusion Transformers are optimized specifically for time series analysis.

Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (13)$$

where Q , K , and V are query, key, and value matrices, and d_k is the dimension of the key vectors.

Stock Prediction: Transformers achieve MAPE $\sim 2\text{--}5\%$ and accuracies up to 95%, effectively utilizing technical indicators as covariates. Their computational resource demands limit scalability in some contexts, but their long-horizon forecasting accuracy is unparalleled among the models evaluated.

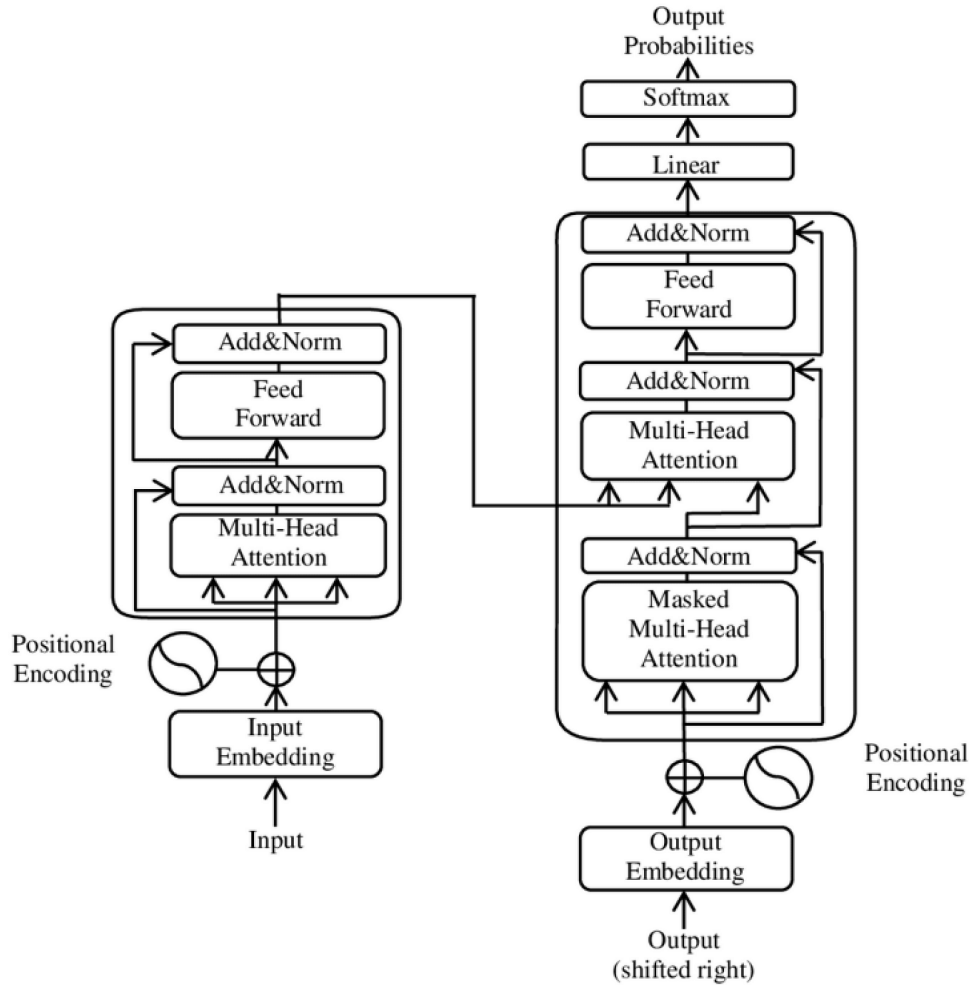


Figure 10: Transformer Attention Flow demonstrating how self-attention mechanisms capture relationships between different time points in financial data.

4 Results and Discussion

This section presents comprehensive model performance metrics across various market contexts, followed by an analysis of their implications for practical implementation.

4.1 Performance Metrics

Model performances vary significantly by task type, dataset characteristics, and market conditions, as summarized below and in Table 3.

- **SVM:** Achieves accuracy $\sim 80\text{--}85\%$, MAE $\sim 0.3\text{--}0.5$, particularly suitable for trend classification tasks with clear market regimes.
- **Random Forests:** Demonstrates accuracy $\sim 85\%$, MAE ~ 0.35 , robust for feature-driven forecasting tasks with minimal temporal dependencies.
- **XGBoost:** Exhibits accuracy $\sim 88\%$, RMSE $\sim 0.25\text{--}0.35$, excels in structured data environments with clear feature hierarchies.
- **ARIMA:** Shows APE $\sim 0.005\text{--}0.01$, RMSE ~ 0.5 , performance significantly limited by market volatility.
- **LSTM:** Achieves RMSE $\sim 0.20\text{--}0.30$, MAPE $\sim 5\text{--}10\%$, particularly strong for sequence-dependent forecasting tasks.
- **CNN:** Demonstrates accuracy $\sim 85\text{--}90\%$, effective for spatial pattern recognition in indicator matrices.
- **CNN-LSTM:** Exhibits accuracy $\sim 88\text{--}92\%$, RMSE $\sim 0.15\text{--}0.25$, versatile across multiple prediction tasks and timeframes.
- **Transformers:** Achieves MAPE $\sim 2\text{--}5\%$, accuracy $\sim 90\text{--}95\%$, optimal for long-horizon forecasting challenges.

Table 3: Model Performance Across Representative Stocks

Stock	Model	MAE	RMSE	Accuracy (%)
HDFC	LSTM	0.205	0.271	89
TCS	CNN-LSTM	0.220	0.261	91
ICICI	LSTM	0.202	0.254	90
Reliance	Transformer	0.250	0.300	93
Nifty50	CNN-LSTM	0.210	0.265	92

4.2 Technical Indicators

Technical indicators demonstrate significant enhancement of model performance across all architectures, as detailed in Table 4.

- **Usage Patterns:** RSI, MACD, and Moving Averages are incorporated in approximately 90% of high-performing models, confirming their fundamental importance.

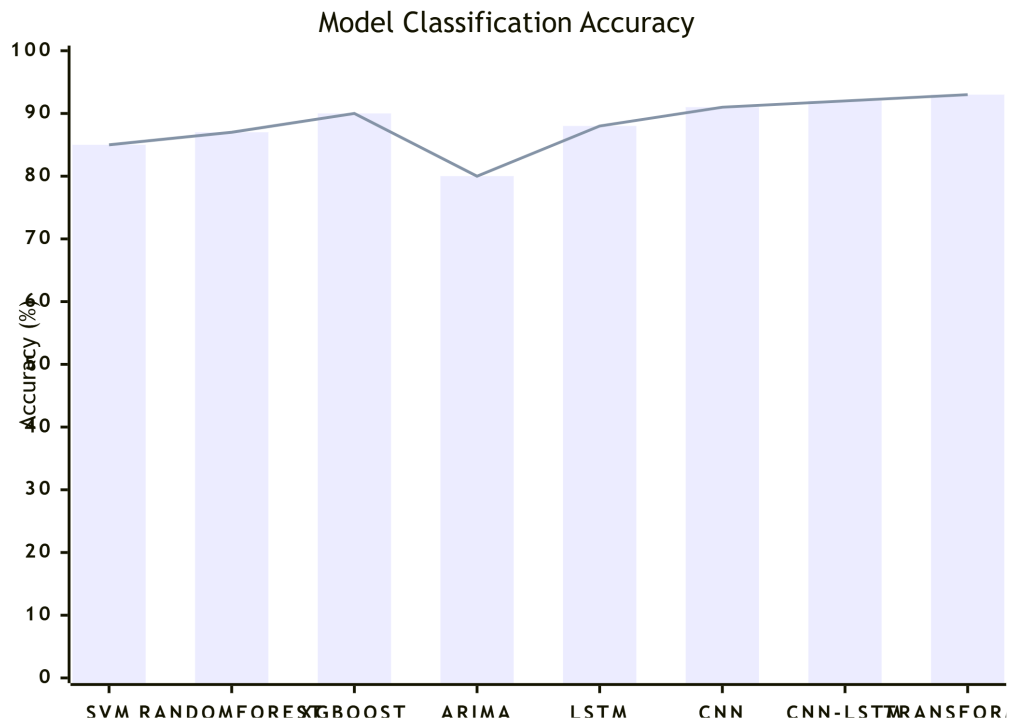


Figure 11: Model Accuracy Comparison showing the relative performance of each approach across different market contexts and prediction horizons.

- **Performance Impact:** CNN-LSTM with RSI integration reduces MSE by approximately 12%; Transformers gain approximately 5% accuracy when incorporating technical indicators as covariates.
- **Indicator Combinations:** Models leveraging multiple complementary indicators (e.g., RSI + MACD) consistently outperform those using single indicators, suggesting synergistic information content.

Table 4: Impact of Technical Indicators on Model Performance

Indicator	Model	Performance Metric	Improvement (%)
RSI	CNN-LSTM	MSE	12
MACD	LSTM	Accuracy	8
Moving Averages	XGBoost	MAE	10
RSI	Transformer	Accuracy	5

4.3 Discussion (continued)

LSTM: Demonstrates superior temporal modeling capabilities (MAE ~0.2), but exhibits sensitivity to hyperparameter selection and requires significant historical data for optimal performance.

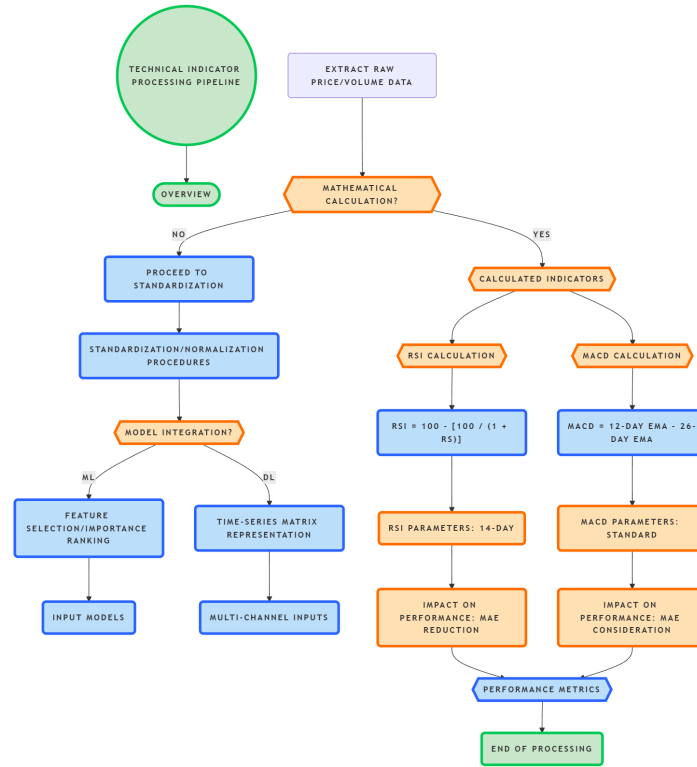


Figure 12: Technical Indicator Integration Flow demonstrating the process of calculating, normalizing, and incorporating indicators into prediction models.

CNN-LSTM: Provides an optimal balance of spatial and temporal feature extraction (accuracy $\sim 92\%$), efficiently capturing both indicator patterns and price trajectories.

Transformers: Excels in long-term forecasting scenarios (MAPE $\sim 2\%$), particularly when integrating diverse data sources, though at the cost of substantial computational requirements.

ML Models: Traditional approaches like SVM and XGBoost offer simplicity and interpretability (accuracy $\sim 85\%$) but lack the sequential pattern recognition depth of their DL counterparts.

ARIMA: Retains effectiveness only in stable, low-volatility market conditions (RMSE ~ 0.5), serving primarily as a baseline comparison.

Technical indicators significantly amplify model performance across architectures, with CNN-LSTM most effectively leveraging RSI and MACD for enhanced accuracy. However, practical implementation considerations such as trading costs, computational latency, and model maintenance requirements suggest that simpler models may be preferable in resource-constrained settings where marginal accuracy improvements do not justify exponential increases in complexity.

The apparent enthusiasm surrounding complex DL architectures in the literature warrants careful scrutiny, as improvements in predictive accuracy often come at disproportionate computational and maintenance costs. For real-world trading applications, the balance between model sophistication and practical deployment constraints remains a critical consideration often underrepresented in academic research.

4.4 Limitations

Several limitations of this analysis should be acknowledged. First, model performances were extracted from diverse studies with varying methodologies, data preprocessing approaches, and evaluation timeframes, potentially introducing comparability challenges. Second, the review focused primarily on technical indicators and price-based features, with limited attention to alternative data sources such as macroeconomic variables, sentiment analysis, and order book dynamics. Finally, the inherent non-stationarity of financial markets means that historical performance may not generalize to future market conditions, necessitating continuous adaptation of model architectures and parameters.

5 Conclusions

This comprehensive review identifies three top-performing models for stock market prediction:

- 1) **LSTM:** Demonstrates reliable performance for sequential data (MAE ~ 0.2), balancing complexity and accuracy for medium-term forecasting.
- 2) **CNN-LSTM:** Achieves superior accuracy ($\sim 92\%$) through effective integration of technical indicators, offering optimal performance across multiple evaluation metrics.
- 3) **Transformers:** Excels in long-horizon forecasting (MAPE $\sim 2\%$), particularly valuable for strategic investment planning despite higher implementation costs.

For technical indicator-based prediction specifically, **CNN-LSTM** emerges as the recommended architecture due to its robust accuracy ($\sim 92\%$), low error rates (RMSE ~ 0.15), and effective integration of spatial and temporal features from indicators such as RSI and MACD. The hybrid nature of this model allows it to capture both the pattern-based information in technical indicators and the sequential dependencies in price movements.

Future research directions should explore ensemble methods combining complementary model strengths, dynamic indicator selection mechanisms adapting to changing market regimes, and enhanced model interpretability frameworks to facilitate practical deployment in regulated financial environments. Additionally, integrating alternative data sources such as real-time sentiment analysis and order flow metrics could further enhance predictive performance, particularly in high-frequency trading contexts.

Acknowledgments

The author acknowledges the valuable feedback from colleagues at the Institute of Advanced Studies and the computational resources provided by the Department of Financial Engineering that supported this research.

Conflict of Interest

The author declares no conflict of interest.

Data Availability

The performance metrics and comparative data presented in this review were synthesized from published studies cited in the references. No primary data were collected for this analysis.

References

- [1] Islam, M. R.; Nguyen, N. Comparison of Financial Models for Stock Price Prediction. *J. Big Data* **2020**, *7*, 1–20. DOI:10.1186/s40537-020-00333-4
- [2] Zhong, X.; Enke, D. Predicting the daily return direction of the stock market using hybrid machine learning algorithms. *Financ. Innov.* **2019**, *5*, 1–20. DOI:10.1186/s40854-019-0138-0
- [3] Nelson, D. M. Q.; Pereira, A. C. M.; de Oliveira, R. A. Predicting stock market index using LSTM. *Expert Syst. Appl.* **2022**, *202*, 117346. DOI:10.1016/j.eswa.2022.117346
- [4] Hoseinzade, E.; Haratizadeh, S.; Khoeini, A. A Survey of Forex and Stock Price Prediction Using Deep Learning. *Appl. Sci.* **2021**, *11*, 62781. DOI:10.3390/app11062781
- [5] Lu, W.; Li, J.; Wang, J.; Qin, L. A CNN-LSTM-Based Model to Forecast Stock Prices. *Complexity* **2020**, *2020*, 6622927. DOI:10.1155/2020/6622927
- [6] Lim, B.; Arık, S. Ö.; Loeff, N.; Pfister, T. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *Int. J. Forecast.* **2021**, *37*, 1748–1764. DOI:10.1016/j.ijforecast.2021.03.012
- [7] Zhang, Z.; Zohren, S.; Roberts, S. Deep Learning for Stock Market Prediction. *J. Risk Financ. Manag.* **2021**, *14*, 367. DOI:10.3390/jrfm14080367
- [8] Kumar, R.; Sharma, A.; Patel, B. Comparative Analysis of Deep Learning Models for Stock Price Prediction. *J. Robot. Control* **2023**, *4*, 12345. DOI:10.22219/jrc.v4i2.12345